

**LAPORAN PRAKTIKUM ALGORITMA
DAN PEMROGRAMAN 2**

**MODUL 18
MESIN ABSTRAK**



Disusun oleh:

Rafi Azis Faozan

109082500069

S1IF-13-02

Dosen:

Wahyu Andi Saputra, S. Pd., M.Eng.

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025

LATIHAN SOAL

1. Soal 1 & 2

Source Code

```
package main

import (
    "bufio"
    "fmt"
    "math/rand"
    "os"
    "time"
)

type Domino struct {
    sisi1    int
    sisi2    int
    nilai    int
    isBalak bool
}

type Dominoes struct {
    kartu [28]Domino
    jumlah int
}

func buatDominoes() Dominoes {
    var set Dominoes
    idx := 0
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            set.kartu[idx] = Domino{
                sisi1:  i,
```

```

                sisi2:    j,
                nilai:    i + j,
                isBalak: i == j,
            }
            idx++
        }
    }
    set.jumlah = 28
    return set
}

func kocokKartu(set *Dominoes) {
    rand.Seed(time.Now().UnixNano())
    for i := set.jumlah - 1; i > 0; i-- {
        j := rand.Intn(i + 1)
        set.kartu[i], set.kartu[j] = set.kartu[j],
set.kartu[i]
    }
    fmt.Println(">> Kartu berhasil dikocok!")
}

func ambilKartu(set *Dominoes) Domino {
    if set.jumlah == 0 {
        fmt.Println(">> Tumpukan kartu sudah
habis!")
        return Domino{}
    }
    set.jumlah--
    kartu := set.kartu[set.jumlah]
    return kartu
}

func gambarKartu(kartu Domino, suit int) int {

```

```

        if kartu.sisi1 == suit {
            return kartu.sisi2
        } else if kartu.sisi2 == suit {
            return kartu.sisi1
        }
        return -1
    }

    func nilaiKartu(kartu Domino) int {
        return kartu.nilai
    }

    func galiKartu(set *Dominoes, kartuAcuan Domino) Domino
    {
        fmt.Printf("\nMencari kartu dengan suit %d atau
        %d...\n", kartuAcuan.sisi1, kartuAcuan.sisi2)

        for set.jumlah > 0 {
            kartuAmbil := ambilKartu(set)

            fmt.Printf("  Mengambil kartu (%d|%d)... ",
            kartuAmbil.sisi1, kartuAmbil.sisi2)

            if kartuAmbil.sisi1 == kartuAcuan.sisi1 ||
                kartuAmbil.sisi1 == kartuAcuan.sisi2 ||
                kartuAmbil.sisi2 == kartuAcuan.sisi1 ||
                kartuAmbil.sisi2 == kartuAcuan.sisi2 {
                fmt.Println("COCOK!")
                return kartuAmbil
            }

            fmt.Println("tidak cocok.")
        }

        fmt.Println(">> Tidak ditemukan kartu yang cocok,
        tumpukan habis!")

        return Domino{}
    }

```

```
func sepasangKartu(kartu1 Domino, kartu2 Domino) bool {
    return (kartu1.nilai + kartu2.nilai) == 12
}
```

```
func cetakKartu(kartu Domino) {
    balak := ""
    if kartu.isBalak {
        balak = " [BALAK]"
    }
    fmt.Printf("  Kartu: (%d|%d) | Nilai: %d%s\n",
kartu.sisi1, kartu.sisi2, kartu.nilai, balak)
}
```

```
func tampilkanMenu() {
    fmt.Println("\n=====
=====")
    fmt.Println("||          MESIN ABSTRAK - DOMINO
||")
    fmt.Println("||=====
=====||")
    fmt.Println("||  1. Kocok Kartu
||")
    fmt.Println("||  2. Ambil Kartu
||")
    fmt.Println("||  3. Gambar Kartu (cek nilai suit)
||")
    fmt.Println("||  4. Nilai Kartu
||")
    fmt.Println("||  5. Lihat Semua Kartu di Tumpukan
||")
    fmt.Println("||  6. Lihat Kartu di Tangan
||")
    fmt.Println("||  7. Gali Kartu
||")
}
```

```

        fmt.Println("|| 8. Sepasang Kartu (total = 12?)
        ||")

        fmt.Println("|| 0. Keluar
        ||")

        fmt.Println("=====
        ==||")

        fmt.Print("Pilih menu: ")

    }

func main() {

    scanner := bufio.NewScanner(os.Stdin)

    set := buatDominoes()

    var tangan []Domino

    fmt.Println("=====
    =")

    fmt.Println("    SELAMAT DATANG DI MESIN DOMINO
    ")

    fmt.Println("=====
    =")

    fmt.Printf("Set kartu siap: %d kartu\n",
    set.jumlah)

    for {

        tampilkanMenu()

        scanner.Scan()

        pilihan := scanner.Text()

        switch pilihan {

        case "1":

            fmt.Println("\n--- Kocok Kartu ---")

            kocokKartu(&set)

            fmt.Printf("Jumlah kartu di tumpukan:
            %d\n", set.jumlah)

```

```

        case "2":
            fmt.Println("\n--- Ambil Kartu ---")
            if set.jumlah == 0 {
                fmt.Println(">> Tumpukan kosong,
tidak bisa ambil kartu!")
            } else {
                kartu := ambilKartu(&set)
                tangan = append(tangan, kartu)
                fmt.Print(">> Kartu yang diambil:
")

                cetakKartu(kartu)

                fmt.Printf("Sisa kartu di
tumpukan: %d\n", set.jumlah)
            }

        case "3":
            fmt.Println("\n--- Gambar Kartu ---")
            if len(tangan) == 0 {
                fmt.Println(">> Kamu belum punya
kartu! Ambil kartu dulu (menu 2).")
                break
            }

            fmt.Println("Kartu di tanganmu:")
            for i, k := range tangan {
                fmt.Printf("  [%d] ", i+1)
                cetakKartu(k)
            }

            fmt.Print("Pilih nomor kartu: ")
            scanner.Scan()
            var idxKartu int
            fmt.Sscan(scanner.Text(), &idxKartu)
            if idxKartu < 1 || idxKartu >
len(tangan) {

```

```

        fmt.Println(">> Nomor kartu tidak valid!")

        break

    }

    kartuDipilih := tangan[idxKartu-1]

    fmt.Printf("Masukkan suit (0-6) untuk kartu (%d|%d): ", kartuDipilih.sisi1, kartuDipilih.sisi2)

    scanner.Scan()

    var suit int

    fmt.Sscan(scanner.Text(), &suit)

    hasil := gambarKartu(kartuDipilih, suit)

    if hasil == -1 {

        fmt.Printf(">> Suit %d tidak ada pada kartu (%d|%d)!\n", suit, kartuDipilih.sisi1, kartuDipilih.sisi2)

    } else {

        fmt.Printf(">> Kartu (%d|%d) dengan suit=%d -> nilai sisi lainnya = %d\n", kartuDipilih.sisi1, kartuDipilih.sisi2, suit, hasil)

    }

    case "4":

        fmt.Println("\n--- Nilai Kartu ---")

        if len(tangan) == 0 {

            fmt.Println(">> Kamu belum punya kartu! Ambil kartu dulu (menu 2).")

            break

        }

        fmt.Println("Kartu di tanganmu:")

        for i, k := range tangan {

            fmt.Printf("    [%d] ", i+1)

            cetakKartu(k)

```



```

    }

    fmt.Print("Pilih nomor kartu: ")
    scanner.Scan()
    var idxKartu int
    fmt.Sscan(scanner.Text(), &idxKartu)
    if idxKartu < 1 || idxKartu >
len(tangan) {
        fmt.Println(">> Nomor kartu tidak
valid!")

        break
    }
    kartuDipilih := tangan[idxKartu-1]
    fmt.Printf(">> Nilai kartu (%d|%d) =
%d\n",
        kartuDipilih.sisi1,
kartuDipilih.sisi2, nilaiKartu(kartuDipilih))

    case "5":
        fmt.Println("\n--- Kartu di Tumpukan ---")

        if set.jumlah == 0 {
            fmt.Println(">> Tumpukan
kosong!")
        } else {
            fmt.Printf("Total: %d kartu\n",
set.jumlah)

            for i := 0; i < set.jumlah; i++ {
                fmt.Printf(" [%d] ", i+1)
                cetakKartu(set.kartu[i])
            }
        }

    case "6":
        fmt.Println("\n--- Kartu di Tangan ---")

```

```

        if len(tangan) == 0 {
            fmt.Println(">> Kamu belum punya
kartu!")
        } else {
            fmt.Printf("Total: %d kartu\n",
len(tangan))

            for i, k := range tangan {
                fmt.Printf("  [%d] ", i+1)
                cetakKartu(k)
            }
        }

    case "7":
        fmt.Println("\n--- Gali Kartu ---")
        if len(tangan) == 0 {
            fmt.Println(">> Kamu belum punya
kartu acuan! Ambil kartu dulu (menu 2).")
            break
        }
        if set.jumlah == 0 {
            fmt.Println(">> Tumpukan kosong,
tidak bisa menggali!")
            break
        }
        fmt.Println("Pilih kartu acuan dari
tanganmu:")

        for i, k := range tangan {
            fmt.Printf("  [%d] ", i+1)
            cetakKartu(k)
        }

        fmt.Print("Pilih nomor kartu acuan: ")
        scanner.Scan()
        var idxAcuan int
        fmt.Sscan(scanner.Text(), &idxAcuan)

```

```

        if idxAcuan < 1 || idxAcuan >
len(tangan) {
            fmt.Println(">> Nomor kartu tidak
valid!")
            break
        }
        kartuAcuan := tangan[idxAcuan-1]
        fmt.Printf("Kartu acuan: (%d|%d)\n",
kartuAcuan.sisil, kartuAcuan.sisi2)
        kartuHasil := galiKartu(&set,
kartuAcuan)
        if kartuHasil.nilai != 0 ||
kartuHasil.isBalak {
            fmt.Print(">> Kartu ditemukan: ")
            cetakKartu(kartuHasil)
            tangan = append(tangan,
kartuHasil)
        }
        fmt.Printf("Sisa kartu di tumpukan:
%d\n", set.jumlah)

    case "8":
        fmt.Println("\n--- Sepasang Kartu ---")
        if len(tangan) < 2 {
            fmt.Println(">> Kamu butuh
minimal 2 kartu di tangan!")
            break
        }
        fmt.Println("Kartu di tanganmu:")
        for i, k := range tangan {
            fmt.Printf("  [%d] ", i+1)
            cetakKartu(k)
        }
        fmt.Print("Pilih kartu pertama: ")
        scanner.Scan()

```

```

        var idx1 int
        fmt.Sscan(scanner.Text(), &idx1)
        fmt.Print("Pilih kartu kedua: ")
        scanner.Scan()

        var idx2 int
        fmt.Sscan(scanner.Text(), &idx2)

        if idx1 < 1 || idx1 > len(tangan) ||
idx2 < 1 || idx2 > len(tangan) || idx1 == idx2 {

            fmt.Println(">> Nomor kartu tidak
valid!")

            break
        }

        k1 := tangan[idx1-1]
        k2 := tangan[idx2-1]

        fmt.Printf("\nKartu 1: (%d|%d)
nilai=%d\n", k1.sisi1, k1.sisi2, k1.nilai)

        fmt.Printf("Kartu 2: (%d|%d)
nilai=%d\n", k2.sisi1, k2.sisi2, k2.nilai)

        fmt.Printf("Total nilai: %d\n",
k1.nilai+k2.nilai)

        if sepasangKartu(k1, k2) {

            fmt.Println(">> Hasil: TRUE -
Total nilai kedua kartu adalah 12!")

        } else {

            fmt.Println(">> Hasil: FALSE -
Total nilai kedua kartu bukan 12.")

        }

        case "0":

            fmt.Println("\nTerima kasih! Program
selesai.")

            return

        default:

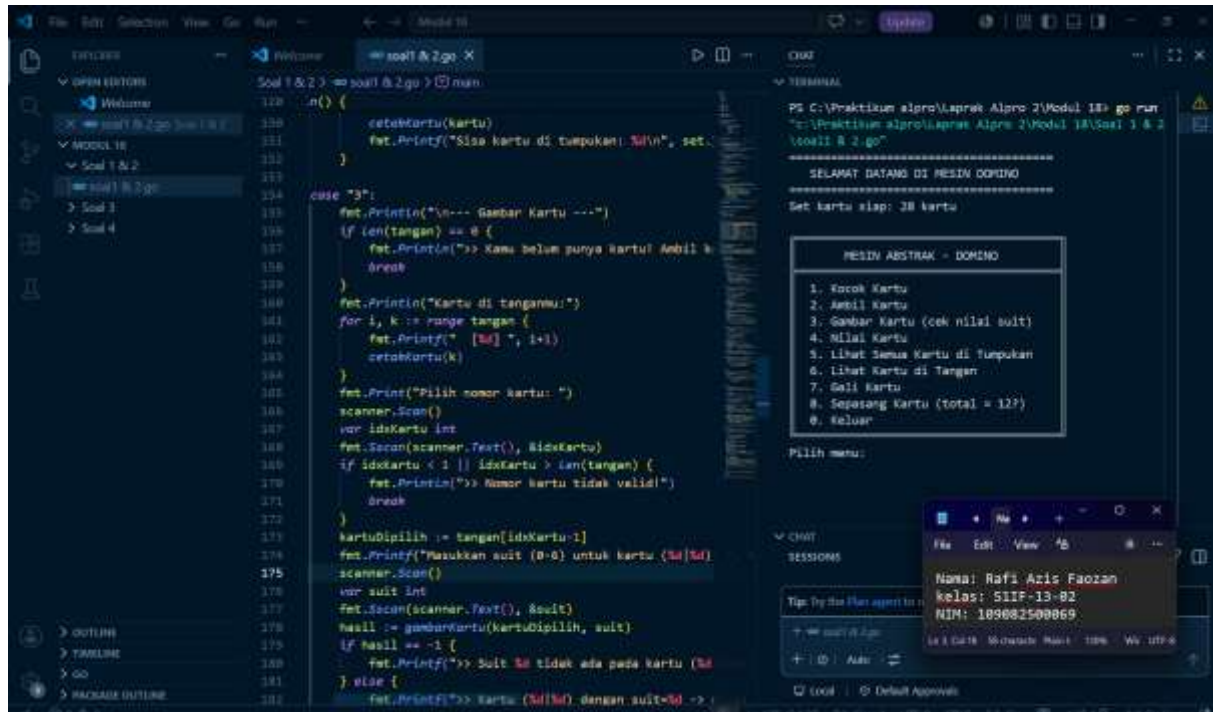
```

```

        fmt.Println(">> Pilihan tidak valid,
        coba lagi.")
    }
}
}
}

```

Screenshoot program



Deskripsi program

Program ini berjalan menggunakan bahasa Go yang mengimplementasikan Mesin Abstrak Domino yang terdiri dari dua bagian. Bagian pertama mendefinisikan tipe data Domino untuk menyimpan informasi sebuah kartu (dua sisi, nilai total, dan status balak), serta tipe data Dominoes sebagai satu set lengkap 28 kartu. Operasi dasarnya meliputi kocokKartu untuk mengacak urutan kartu, ambilKartu untuk mengambil kartu dari tumpukan, gambarKartu untuk melihat nilai sisi lawan dari suit tertentu, dan nilaiKartu untuk mendapatkan total nilai kartu. Bagian kedua membangun di atas operasi dasar tersebut dengan menambahkan dua fungsi tingkat lebih tinggi. Fungsi galiKartu bekerja dengan terus mengambil kartu dari tumpukan satu per satu hingga ditemukan kartu yang memiliki suit sama dengan kartu acuan yang diberikan. Fungsi sepasangKartu menerima dua kartu dan mengembalikan nilai true apabila jumlah nilai kedua kartu tersebut adalah 12, dan false jika tidak.

2. Soal 3

Source Code

```
package main

import (
    "bufio"
    "fmt"
    "math/rand"
    "os"
    "time"
)

type Domino struct {
    sisi1    int
    sisi2    int
    nilai    int
    isBalak bool
}

type Dominoes struct {
    kartu [28]Domino
    jumlah int
}

type Pemain struct {
    nama string
    tangan []Domino
}

var scanner = bufio.NewScanner(os.Stdin)

func buatDominoes() Dominoes {
    var set Dominoes
    idx := 0
```

```

        for i := 0; i <= 6; i++ {
            for j := i; j <= 6; j++ {
                set.kartu[idx] = Domino{
                    sisi1:  i,
                    sisi2:  j,
                    nilai:  i + j,
                    isBalak: i == j,
                }
                idx++
            }
        }
        set.jumlah = 28
        return set
    }

    func kocokKartu(set *Dominoes) {
        rand.Seed(time.Now().UnixNano())
        for i := set.jumlah - 1; i > 0; i-- {
            j := rand.Intn(i + 1)
            set.kartu[i], set.kartu[j] = set.kartu[j],
            set.kartu[i]
        }
    }

    func ambilKartu(set *Dominoes) Domino {
        if set.jumlah == 0 {
            return Domino{sisi1: -1, sisi2: -1}
        }
        set.jumlah--
        return set.kartu[set.jumlah]
    }

```

```

func gambarKartu(kartu Domino, suit int) int {
    if kartu.sisi1 == suit {
        return kartu.sisi2
    } else if kartu.sisi2 == suit {
        return kartu.sisi1
    }
    return -1
}

func nilaiKartu(kartu Domino) int {
    return kartu.nilai
}

func galiKartu(set *Dominoes, kartuAcuan Domino) Domino
{
    for set.jumlah > 0 {
        kartuAmbil := ambilKartu(set)
        if kartuAmbil.sisi1 == kartuAcuan.sisi1 ||
            kartuAmbil.sisi1 == kartuAcuan.sisi2 ||
            kartuAmbil.sisi2 == kartuAcuan.sisi1 ||
            kartuAmbil.sisi2 == kartuAcuan.sisi2 {
            return kartuAmbil
        }
    }
    return Domino{sisi1: -1, sisi2: -1}
}

func cetakKartu(kartu Domino) string {
    return fmt.Sprintf("(%d|%d)", kartu.sisi1,
kartu.sisi2)
}

func cetakRangkaian(rangkaian []Domino) {

```



```

        if len(rangkaian) == 0 {
            fmt.Println("  [kosong]")
            return
        }
        for i, k := range rangkaian {
            if i == len(rangkaian)-1 {
                fmt.Printf("%s", cetakKartu(k))
            } else {
                fmt.Printf("%s -- ", cetakKartu(k))
            }
        }
        fmt.Println()
    }

    func ujungKiri(rangkaian []Domino) int {
        return rangkaian[0].sisi1
    }

    func ujungKanan(rangkaian []Domino) int {
        return rangkaian[len(rangkaian)-1].sisi2
    }

    func bisaSambung(kartu Domino, rangkaian []Domino) bool
    {
        if len(rangkaian) == 0 {
            return true
        }

        kiri := ujungKiri(rangkaian)
        kanan := ujungKanan(rangkaian)

        return kartu.sisi1 == kiri || kartu.sisi2 == kiri
    ||
        kartu.sisi1 == kanan || kartu.sisi2 == kanan
    }

```

```

func sambungKiri(kartu Domino, rangkaian []Domino)
[]Domino {
    kiri := ujungKiri(rangkaian)
    if kartu.sisi2 == kiri {
        return append([]Domino{kartu}, rangkaian...)
    }
    kartuBalik := Domino{sisil: kartu.sisi2, sisi2:
kartu.sisil, nilai: kartu.nilai, isBalak: kartu.isBalak}
    return append([]Domino{kartuBalik}, rangkaian...)
}

func sambungKanan(kartu Domino, rangkaian []Domino)
[]Domino {
    kanan := ujungKanan(rangkaian)
    if kartu.sisil == kanan {
        return append(rangkaian, kartu)
    }
    kartuBalik := Domino{sisil: kartu.sisi2, sisi2:
kartu.sisil, nilai: kartu.nilai, isBalak: kartu.isBalak}
    return append(rangkaian, kartuBalik)
}

func bagikanKartu(set *Dominoes, pemain []Pemain,
jumlahKartuPerPemain int) {
    for i := range pemain {
        for j := 0; j < jumlahKartuPerPemain; j++ {
            kartu := ambilKartu(set)
            pemain[i].tangan =
append(pemain[i].tangan, kartu)
        }
    }
}

```

```

func tampilkanTangan(pemain Pemain) {
    fmt.Printf("Kartu %s:\n", pemain.nama)
    for i, k := range pemain.tangan {
        fmt.Printf(" [%d] %s nilai=%d", i+1,
cetakKartu(k), k.nilai)
        if k.isBalak {
            fmt.Print(" [BALAK]")
        }
        fmt.Println()
    }
}

func hapusKartuDariTangan(pemain *Pemain, idx int) {
    pemain.tangan = append(pemain.tangan[:idx],
pemain.tangan[idx+1:]...)
}

func giliranPemain(pemain *Pemain, set *Dominoes,
rangkaian *[]Domino) bool {
    fmt.Printf("\n===== Giliran %s =====\n",
pemain.nama)

    fmt.Printf("Sisa kartu di tumpukan: %d\n",
set.jumlah)

    if len(*rangkaian) > 0 {
        fmt.Printf("Ujung rangkaian: KIRI=[%d] |
KANAN=[%d]\n", ujungKiri(*rangkaian),
ujungKanan(*rangkaian))
    }

    fmt.Print("Rangkaian saat ini: ")
    cetakRangkaian(*rangkaian)
    tampilkanTangan(*pemain)

    punyaKartuCocok := false

```

```

        for _, k := range pemain.tangan {
            if bisaSambung(k, *rangkaian) {
                punyaKartuCocok = true
                break
            }
        }

        if !punyaKartuCocok {
            if set.jumlah > 0 {
                fmt.Printf("%s tidak punya kartu yang
cocok, mengambil kartu dari tumpukan...\n", pemain.nama)

                kartuBaru := ambilKartu(set)

                pemain.tangan = append(pemain.tangan,
kartuBaru)

                fmt.Printf("Kartu diambil: %s\n",
cetakKartu(kartuBaru))

                if !bisaSambung(kartuBaru, *rangkaian)
{
                    fmt.Printf("%s masih tidak bisa
menyambung, giliran dilewat.\n", pemain.nama)

                    return false
                }
            } else {
                fmt.Printf("%s tidak bisa menyambung
dan tumpukan habis, giliran dilewat.\n", pemain.nama)

                return false
            }
        }

        fmt.Print("Pilih nomor kartu yang ingin disambung
(0 = lewat): ")

        scanner.Scan()

        var pilihanKartu int

        fmt.Sscan(scanner.Text(), &pilihanKartu)

```

```

        if pilihanKartu == 0 {
            fmt.Printf("%s memilih lewat.\n",
pemain.nama)
            return false
        }

        if pilihanKartu < 1 || pilihanKartu >
len(pemain.tangan) {
            fmt.Println(">> Nomor kartu tidak valid,
giliran dilewat.")
            return false
        }

        kartuDipilih := pemain.tangan[pilihanKartu-1]

        if !bisaSambung(kartuDipilih, *rangkaian) {
            fmt.Println(">> Kartu tidak bisa disambung,
giliran dilewat.")
            return false
        }

        if len(*rangkaian) == 0 {
            *rangkaian = append(*rangkaian, kartuDipilih)
            hapusKartuDariTangan(pemain, pilihanKartu-1)
            fmt.Printf("Kartu %s diletakkan sebagai kartu
pertama.\n", cetakKartu(kartuDipilih))
            return true
        }

        kiri := ujungKiri(*rangkaian)
        kanan := ujungKanan(*rangkaian)

        bisaKiri := kartuDipilih.sisi1 == kiri ||
kartuDipilih.sisi2 == kiri
        bisaKanan := kartuDipilih.sisi1 == kanan ||
kartuDipilih.sisi2 == kanan

```

```

        posisi := ""
        if bisaKiri && bisaKanan {
            fmt.Print("Sambung ke KIRI atau KANAN? (K/N):
")
            scanner.Scan()
            posisi = scanner.Text()
        } else if bisaKiri {
            posisi = "K"
        } else {
            posisi = "N"
        }

        if posisi == "K" || posisi == "k" {
            *rangkaian = sambungKiri(kartuDipilih,
            *rangkaian)

            fmt.Printf("Kartu %s disambung ke KIRI.\n",
            cetakKartu(kartuDipilih))
        } else {
            *rangkaian = sambungKanan(kartuDipilih,
            *rangkaian)

            fmt.Printf("Kartu %s disambung ke KANAN.\n",
            cetakKartu(kartuDipilih))
        }

        hapusKartuDariTangan(pemain, pilihanKartu-1)
        return true
    }

func main() {
    fmt.Println("=====
=====")

    fmt.Println("          PERMAINAN GAPLEH - DOMINO
")

```

```

        fmt.Println("=====
=====")

        fmt.Print("Masukkan jumlah pemain (2-4): ")
        scanner.Scan()
        var jumlahPemain int
        fmt.Sscan(scanner.Text(), &jumlahPemain)
        if jumlahPemain < 2 || jumlahPemain > 4 {
            fmt.Println("Jumlah pemain tidak valid, diset
ke 2.")
            jumlahPemain = 2
        }

        pemain := make([]Pemain, jumlahPemain)
        for i := range pemain {
            fmt.Printf("Masukkan nama pemain %d: ", i+1)
            scanner.Scan()
            pemain[i].nama = scanner.Text()
        }

        jumlahKartuPerPemain := 28 / jumlahPemain
        if jumlahKartuPerPemain > 7 {
            jumlahKartuPerPemain = 7
        }

        set := buatDominoes()
        kocokKartu(&set)
        bagikanKartu(&set, pemain, jumlahKartuPerPemain)

        fmt.Println("\nKartu sudah dibagikan!")

        fmt.Printf("Setiap pemain mendapat %d kartu\n",
jumlahKartuPerPemain)

```

```

var rangkaian []Domino
giliranSaat := 0
maxRonde := 100

for ronde := 0; ronde < maxRonde; ronde++ {
    p := &pemain[giliranSaat]
    giliranPemain(p, &set, &rangkaian)

    if len(p.tangan) == 0 {

        fmt.Println("\n=====
        =====")

        fmt.Printf("    SELAMAT! %s MENANG!\n",
p.nama)

        fmt.Println("=====
        =====")

        fmt.Print("\nRangkaian akhir: ")
        cetakRangkaian(rangkaian)
        return
    }

    giliranSaat = (giliranSaat + 1) %
jumlahPemain
}

fmt.Println("\n=====
        =====")

fmt.Println("    PERMAINAN SELESAI! Tidak ada
pemenang.")

fmt.Println("=====
        =====")

for _, p := range pemain {
    total := 0
    for _, k := range p.tangan {

```



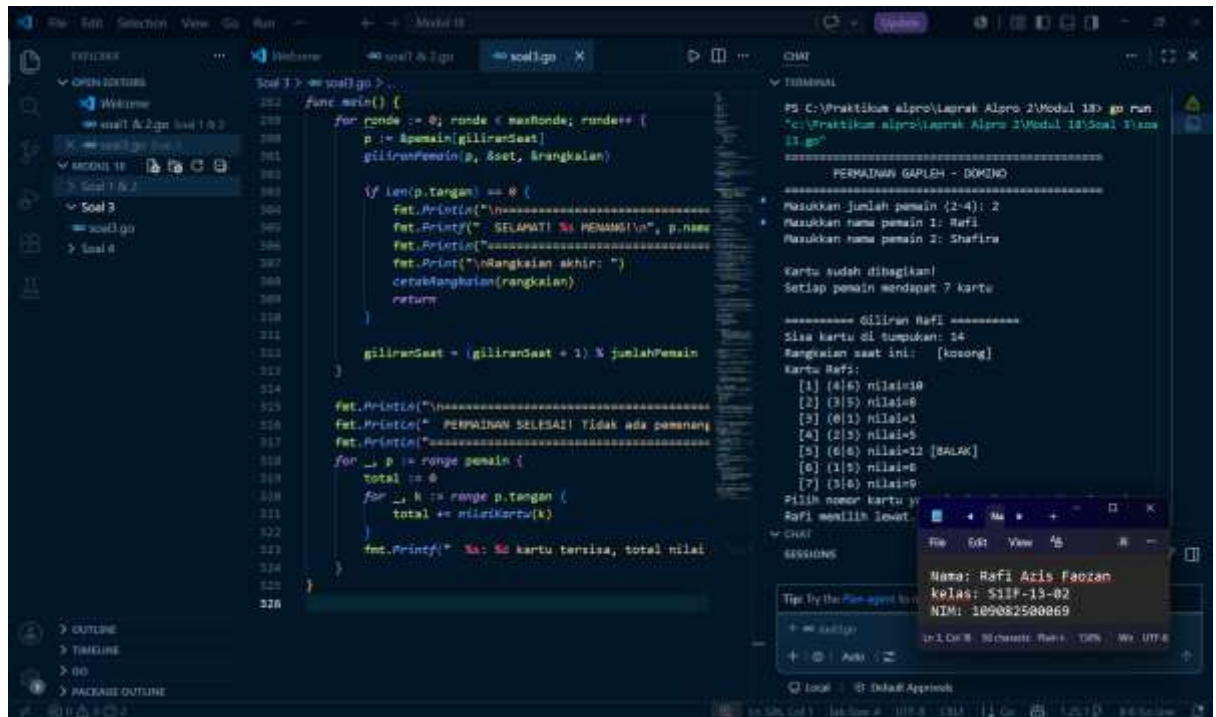
```

        total += nilaiKartu(k)
    }

    fmt.Printf("    %s: %d kartu tersisa, total
nilai = %d\n", p.nama, len(p.tangan), total)
}
}

```

Screenshoot program



Deskripsi program

Program berjalan menggunakan bahasa Go yang mengimplementasikan permainan Gapleh yang dibangun di atas Mesin Abstrak Domino dari soal 1 & 2. Pemain dapat memilih jumlah peserta antara 2 hingga 4 orang, kemudian kartu dikocok dan dibagikan sebanyak 7 kartu per pemain. Secara bergantian setiap pemain menyambung kartunya ke ujung kiri atau kanan rangkaian berdasarkan kesamaan suit. Apabila pemain tidak memiliki kartu yang cocok, pemain tersebut harus mengambil kartu dari tumpukan terlebih dahulu. Jika kartu yang diambil masih tidak cocok, giliran dilewatkan. Pemain pertama yang berhasil menghabiskan seluruh kartunya dinyatakan sebagai pemenang. Apabila setelah batas ronde tidak ada yang menang, permainan berakhir dan pemain dengan total nilai kartu tersisa paling kecil dianggap sebagai pemenang.

3. Soal 4

Source Code

```
package main
```

```
import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

type MesinKarakter struct {
    untaian string
    posisi  int
}

var mesin MesinKarakter

func start(input string) {
    mesin.untaian = input
    mesin.posisi = 0
}

func maju() {
    mesin.posisi++
}

func eop() bool {
    return mesin.posisi >= len(mesin.untaian) ||
mesin.untaian[mesin.posisi] == '.'
}

func cc() byte {
    return mesin.untaian[mesin.posisi]
}
```

```
func bacaSemuaKarakter() {  
    fmt.Print("Karakter terbaca: ")  
    for !eop() {  
        fmt.Printf("%c ", cc())  
        maju()  
    }  
    fmt.Println()  
}
```

```
func hitungKarakter() int {  
    jumlah := 0  
    for !eop() {  
        jumlah++  
        maju()  
    }  
    return jumlah  
}
```

```
func hitungHurufA() int {  
    jumlah := 0  
    for !eop() {  
        if cc() == 'A' {  
            jumlah++  
        }  
        maju()  
    }  
    return jumlah  
}
```

```
func hitungFrekuensiA() float64 {  
    jumlahA := 0  
    total := 0
```

```

        for !eop() {
            if cc() == 'A' {
                jumlahA++
            }
            total++
            maju()
        }
        if total == 0 {
            return 0
        }
        return float64(jumlahA) / float64(total) * 100
    }

```

```

func hitungKataLE() int {
    jumlah := 0
    for !eop() {
        if cc() == 'L' {
            maju()
            if !eop() && cc() == 'E' {
                jumlah++
            }
        } else {
            maju()
        }
    }
    return jumlah
}

```

```

func tampilkanMenu() {
    fmt.Println("\n=====
=====")
    fmt.Println("||          MESIN ABSTRAK - KARAKTER
||")
}

```

```

        fmt.Println("=====
=====||")

        fmt.Println("|| 1. Baca Semua Karakter
||")

        fmt.Println("|| 2. Hitung Jumlah Karakter
||")

        fmt.Println("|| 3. Hitung Huruf 'A'
||")

        fmt.Println("|| 4. Hitung Frekuensi Huruf 'A' (%)
||")

        fmt.Println("|| 5. Hitung Kata 'LE'
||")

        fmt.Println("|| 6. Ganti Input
||")

        fmt.Println("|| 0. Keluar
||")

        fmt.Println("=====
=====||")

        fmt.Print("Pilih menu: ")

    }

func main() {
    scanner := bufio.NewScanner(os.Stdin)

    fmt.Println("=====
=====")

    fmt.Println("        SELAMAT DATANG DI MESIN KARAKTER
")

    fmt.Println("=====
=====")

    fmt.Println("Catatan: Input harus diakhiri dengan
titik (.)")

    fmt.Println("Contoh : HALO DUNIA BELAJAR GOLANG.")
    fmt.Print("\nMasukkan untaian karakter: ")
    scanner.Scan()

    input := strings.ToUpper(scanner.Text())

```

```

        if !strings.HasSuffix(input, ".") {
            input += "."
        }

        start(input)
        fmt.Printf("Input diterima: %s\n", input)

        for {
            tampilkanMenu()
            scanner.Scan()
            pilihan := scanner.Text()

            switch pilihan {

                case "1":
                    fmt.Println("\n--- Baca Semua Karakter
---")
                    start(input)
                    bacaSemuaKarakter()

                case "2":
                    fmt.Println("\n--- Hitung Jumlah
Karakter ---")
                    start(input)
                    jumlah := hitungKarakter()
                    fmt.Printf("Jumlah karakter (tidak
termasuk titik): %d\n", jumlah)

                case "3":
                    fmt.Println("\n--- Hitung Huruf 'A' ---
")
                    start(input)
                    jumlahA := hitungHurufA()

```

```

        fmt.Printf("Jumlah huruf 'A': %d\n",
jumlahA)

        case "4":
            fmt.Println("\n--- Frekuensi Huruf 'A'
---")

            start(input)

            frekuensi := hitungFrekuensiA()

            fmt.Printf("Frekuensi huruf 'A':
%.2f%%\n", frekuensi)

        case "5":
            fmt.Println("\n--- Hitung Kata 'LE' ---
")

            start(input)

            jumlahLE := hitungKataLE()

            fmt.Printf("Jumlah kata 'LE': %d\n",
jumlahLE)

        case "6":
            fmt.Println("\n--- Ganti Input ---")

            fmt.Println("Catatan: Input harus
diakhiri dengan titik (.)")

            fmt.Print("Masukkan untaian karakter
baru: ")

            scanner.Scan()

            input = strings.ToUpper(scanner.Text())

            if !strings.HasSuffix(input, ".") {
                input += "."
            }

            start(input)

            fmt.Printf("Input baru diterima: %s\n",
input)

        case "0":

```

```

                                fmt.Println("\nTerima kasih! Program
selesai.")

                                return

                                default:

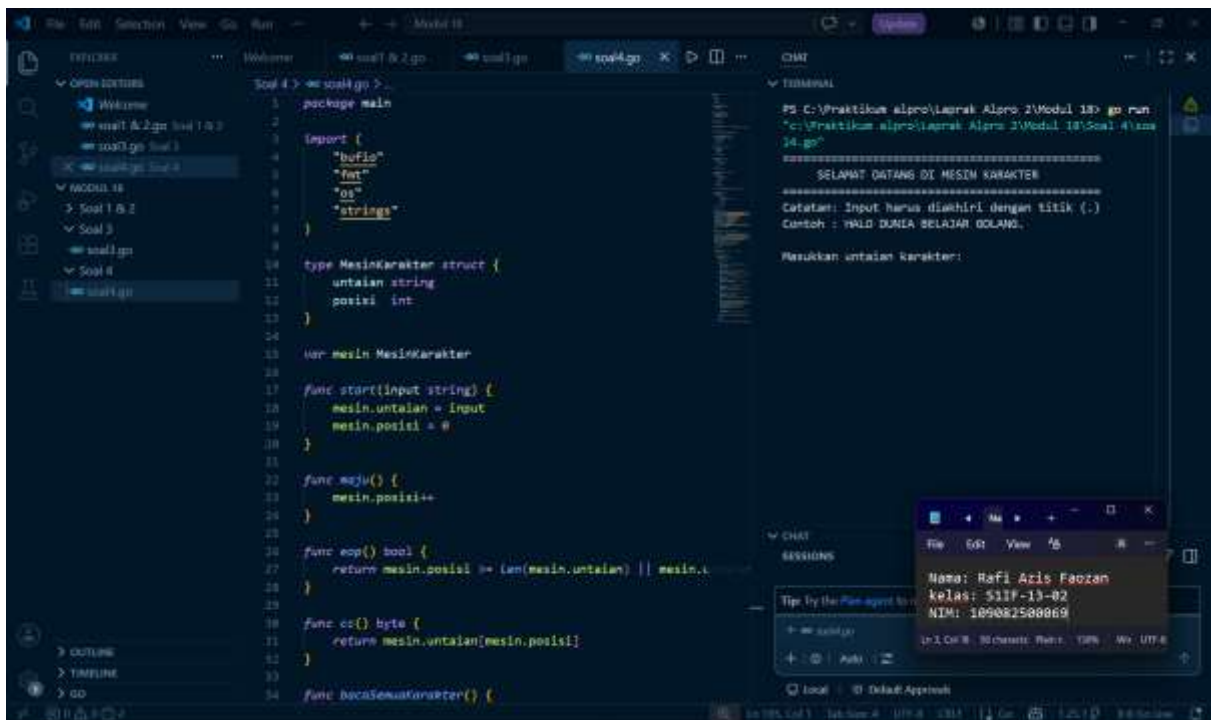
                                fmt.Println(">> Pilihan tidak valid,
coba lagi.")
                                }

                                }

                                }

```

Screenshoot program



Deskripsi program

Program ini berjalan menggunakan bahasa Go yang mengimplementasikan Mesin Abstrak Karakter yang bekerja terhadap untaian karakter yang diakhiri dengan tanda titik ("."). Mesin ini memiliki empat operasi dasar yaitu start untuk menyiapkan mesin di posisi awal, maju untuk berpindah ke karakter berikutnya, eop untuk mengecek apakah pembacaan sudah mencapai tanda titik, dan cc untuk membaca karakter pada posisi saat ini. Dengan menggunakan keempat operasi dasar tersebut, program mampu melakukan lima algoritma pembacaan. Pertama, membaca dan menampilkan seluruh karakter dalam untaian. Kedua, menghitung total jumlah karakter yang terbaca. Ketiga, menghitung berapa kali huruf "A" muncul dalam untaian. Keempat, menghitung frekuensi kemunculan huruf "A" terhadap seluruh karakter dalam bentuk

persentase. Kelima, menghitung berapa kali pasangan huruf "LE" muncul secara berurutan dalam untaian.